■ Exercise — Sales Ledger Analyzer (Batch 1)

Goal

Build a Python program that ingests a CSV of transactions, validates each row, aggregates the valid data, and then writes:
1) a **JSON summary report**, and
2) an **errors log** listing all rejected rows and why they were rejected.
This exercise tests: file I/O, functions, loops, conditionals, collections, try/except, and comprehensions.

Input format

CSV with header:

```
date,customer,category,amount
YYYY-MM-DD,STRING,STRING,FLOAT
```

What your program must do

- **1) Read & validate:** check each row for valid date, non-empty customer/category, numeric amount. Invalid rows → errors.log.
- **2) Aggregate:** from valid rows, compute totals by customer and by category (round to 2 decimals).
- **3) Report:** create report.json with totals, top customer/category, valid transaction count, invalid row count.
- **4) Use required functions:** implement load_transactions, validate_row, compute_totals, pick_top, write_report.
- **5) Error handling:** use try/except around file I/O and parsing.
- **6) Collections & comprehensions:** use dicts/lists, at least one comprehension.

Output example

```
{
  "totals": {
    "by_customer": { "Alice": 313.89, "Bob": 64.8, "Charlie": 0.0 },
    "by_category": { "groceries": 98.2, "books": 30.5, "electronics": 249.99 }
  },
  "top_customer": "Alice",
  "top_category": "electronics",
  "transactions_count": 6,
  "invalid_rows": 2
}
```

Errors log example

```
Line 5: invalid amount 'abc'
Line 6: missing customer
```

How to run & submit

- Place your script (e.g., analyzer.py) alongside the CSV.
- Run it against the provided file.
- Commit in a branch named *lesson-exam* (or similar) and share the repository URL.